

챕터 3. 도메인을 중심으로 - DDD + 클린 아키텍처와 Kubernetes

이 챕터의 전체 소스코드는 <https://github.com/metacoding-12-msa/ex02> 에서 확인할 수 있습니다.

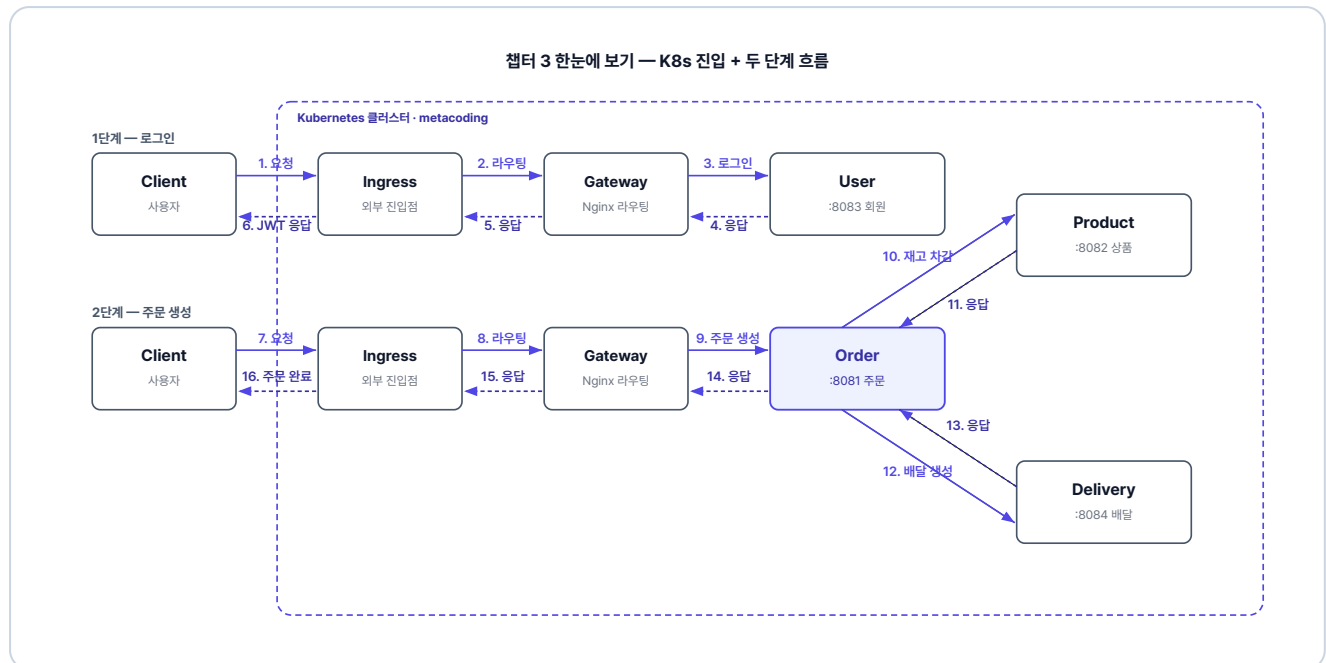


그림 3-1. 챕터 3 한눈에 보기 - K8s 진입과 두 단계 흐름

이번 챕터가 끝나면

- 비즈니스 규칙을 도메인 객체에 캡슐화하는 **DDD(도메인 주도 개발)** 를 적용할 수 있습니다.
- 컨트롤러가 `OrderService` 코드 대신 UseCase 인터페이스에 의존하도록 **클린 아키텍처**로 구조를 정리할 수 있습니다.
- Nginx API Gateway로 4개 서비스 진입점을 하나로 통합할 수 있습니다.
- Kubernetes에 매니페스트로 배포하고 ConfigMap·Secret으로 환경 변수를 주입할 수 있습니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/ex02.git
cd ex02
```

2. 파일 구조

ex02 디렉토리

```
ex02/
├─ order/           # 포트 8081
├─ product/         # 포트 8082
├─ user/            # 포트 8083
├─ delivery/        # 포트 8084
├─ gateway/         # Nginx API Gateway
├─ db/              # MySQL Dockerfile
└─ k8s/             # Kubernetes 매니페스트
```

주문 서비스 패키지 구조 (챕터 3에서 재구성)

```
src/main/java/com/metacoding/order/
├─ domain/          # 엔티티 + 비즈니스 규칙
├─ repository/      # Spring Data JPA
├─ usecase/         # UseCase 인터페이스 + 서비스 코드
├─ web/             # 컨트롤러 + DTO
├─ adapter/         # 외부 서비스 클라이언트 (order 전용)
└─ core/            # JWT, 예외처리 (챕터 2와 동일)
src/main/resources/
└─ application.properties # DB·JWT 설정 (값은 환경변수로 주입)
```

user/product/delivery도 동일한 구조이며, adapter/ 패키지만 order 전용입니다.

3. 실습 환경

도구	용도	비고
Docker Desktop	컨테이너 런타임	챕터 2에서 설치한 그대로. 실행 중이어야 함
Minikube	로컬 Kubernetes 클러스터	https://minikube.sigs.k8s.io/

Docker Desktop이 "Engine running" 상태인지 확인하고, Minikube가 설치되어 있지 않다면 위 주소에서 설치합니다.

4. 실습 순서

1. 챕터 2 코드를 DDD + 클린 아키텍처로 재구성한 결과 살펴보기
2. Nginx API Gateway 살펴보기
3. K8s 매니페스트(ConfigMap·Secret·Deployment·Service·Ingress) 5종 살펴보기
4. Minikube에서 빌드·배포·실행

이번 챕터는 직접 코드를 작성하지 않습니다. 챕터 2 프로젝트를 DDD + 클린 아키텍처로 재구성한 결과를 살펴보고, Kubernetes 배포를 실습합니다.

팀장이 코드 정리를 부탁했습니다.

팀장 : "취소되는지 검사하는 코드가 서비스마다 흩어져 있어요. 손볼 때마다 어디를 고쳐야 할지 헷갈리니까, 한곳으로 모아 주세요."

오픈이가 `OrderService`의 주문 취소 메서드를 열었습니다. "이미 취소된 주문인가?"를 막는 `if`가 거기 있었습니다. 그런데 코드를 더 훑어보니 같은 종류의 검증이 서비스마다 박혀 있습니다. 재고가 충분한지는 `ProductService`의 `if`가, 배달 주소가 비어 있는지는 `DeliveryService`의 `if`가 따로 판단합니다. `Order`는 데이터만 담고 있을 뿐, 취소 가능 여부 판단은 모두 바깥 서비스에 흩어져 있습니다.

함께 수정 작업을 하던 동료와 와서 코드를 들여다봤습니다.

동료 : "근데 이상하지 않아요? `Order` 안에 데이터는 다 있는데, 취소 여부는 `OrderService`가 결정하잖아요. 서비스는 그냥 흐름만 처리하고, 검증은 도메인에서 하는 게 낫지 않을까요?"

그 말을 들은 오픈이는 선배 자리로 가 물었습니다.

선배 : "맞아요. 비즈니스 로직은 도메인에 두고, 서비스는 흐름만 처리하는 게 좋아요. 그 방식이 **DDD(도메인 주도 개발)**예요. 이렇게 하면 흩어져 있던 검증이 도메인 한곳에 모여서, 다음에 손볼 때도 도메인만 보면 됩니다."

DDD(도메인 주도 개발)란? 비즈니스 규칙(검증·상태 변경 같은 핵심 로직)을 도메인 객체 안에 두는 설계 방식입니다. 데이터와 규칙을 같은 객체에 모아, 서비스 계층은 흐름 조율에만 집중하게 합니다.

3.1 도메인 주도 개발(Domain-Driven Design) - 비즈니스 로직을 도메인으로

가게 운영 규칙이 사장 한 명의 머릿속에만 있다고 해보겠습니다. 환불 가능 시간, 결제 방식, 재고 처리까지 전부 사장이 외우고 있습니다. 그래서 누가 손님을 응대하든 사장이 대답을 해야 합니다.

이 문제는 가게 운영 매뉴얼을 만들면 해결됩니다. 규칙은 매뉴얼에 정리해 두고, 사장은 손님 응대 흐름만 진행하면서 매뉴얼에 적힌 대로 따릅니다. 누가 응대하든 매뉴얼만 보면 같은 판단을 할 수 있습니다. 새 규칙이 들어와도 매뉴얼 한 곳만 업데이트하면 끝입니다.



그림 3-2. 머릿속 규칙에서 운영 매뉴얼로

`OrderService`가 비즈니스를 수행하는 '사장님'이라면, `Order` 도메인 객체는 그 안에 담긴 핵심 규칙인 '운영 매뉴얼'입니다. 비즈니스 로직을 서비스에서 분리해 도메인에 응집시키면 기술적 환경이 변해도 비즈니스의 본질은 흔들리지 않으며, 복잡한 요구사항 속에서도 코드의 가독성과 유지보수성을 지킬 수 있습니다.

3.2 UseCase 인터페이스(Use Case Interface) - USB 허브처럼 갈아끼우기

`Order`가 규칙을 갖게 됐어도, 컨트롤러가 `OrderService` 코드를 직접 참조하면 그 코드가 바뀔 때마다 컨트롤러도 바뀌어야 하고, 테스트 때 가짜 `OrderService`로 갈아끼우기도 어렵습니다.

USB 허브를 떠올려 보세요. USB 규격만 같으면 마우스든 키보드든 외장하드든 같은 허브에 꽂아 쓸 수 있고, 필요에 따라 갈아끼웁니다. 컴퓨터는 어떤 장치가 꽂혔는지 모르고, USB 규격대로 통신만 합니다.

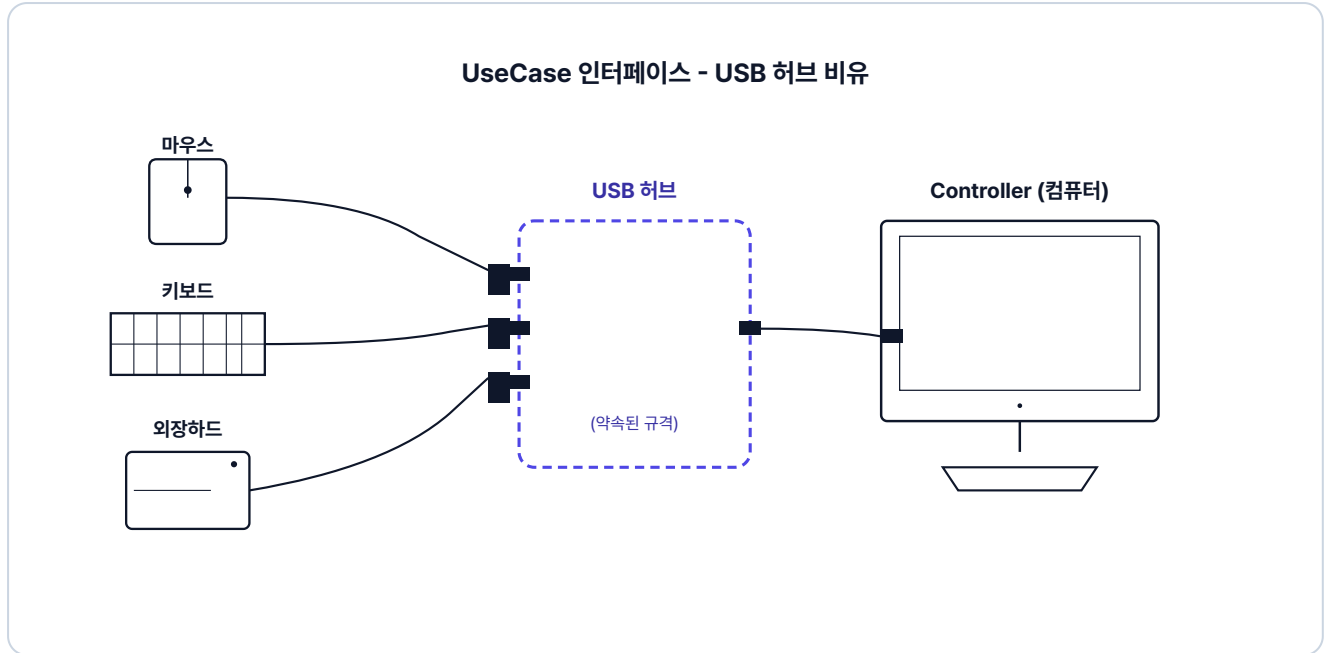


그림 3-3. UseCase 인터페이스 - USB 허브 비유

UseCase 인터페이스가 컨트롤러와 약속한 'USB 규격'이라면, `OrderService`는 그 규격에 꽂히는 'USB 장치'입니다. 규격만 지키면 장치를 자유롭게 갈아끼울 수 있고, 컴퓨터(컨트롤러)는 어떤 장치가 꽂혔는지 알 필요가 없습니다.

UseCase 인터페이스란? 시스템이 수행할 비즈니스 행위(주문 생성·조회·취소 같은)를 메서드로 약속한 인터페이스입니다. 컨트롤러는 이 약속만 보고 호출하므로, 뒤에 어떤 코드가 꽂혀도 동작합니다.

컨트롤러도 **UseCase 인터페이스**(USB 규격)만 보고, 뒤에 어떤 `OrderService` 코드가 꽂혔는지는 모르게 만듭니다. 약속에만 의존하는 이 방식이 **클린 아키텍처(Clean Architecture)**의 의존성 역전입니다.

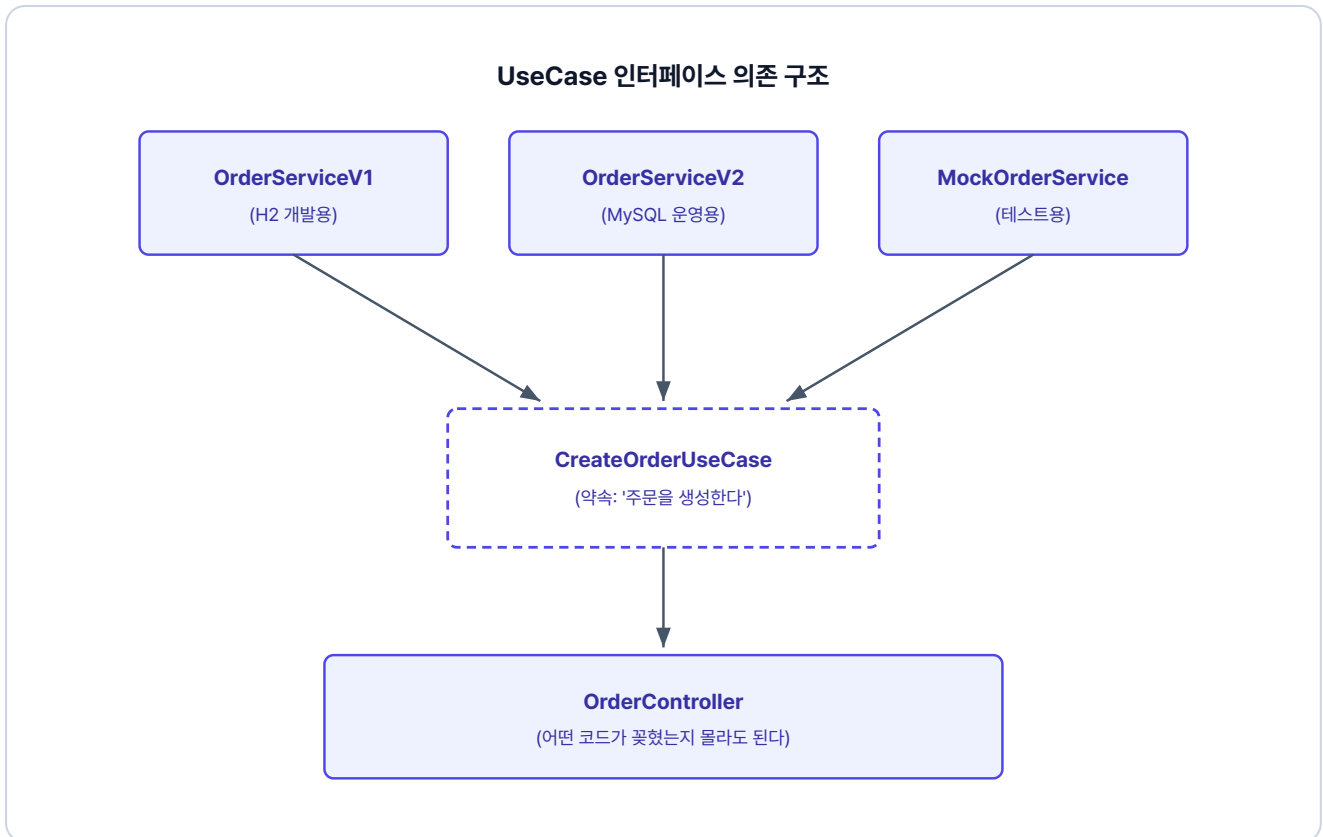


그림 3-4. UseCase 인터페이스 의존 구조

"무엇을 할 것인가"(UseCase 인터페이스)와 "어떻게 할 것인가"(Service 코드)를 분리하는 것이 핵심입니다.

이 책에서는 클린 아키텍처의 핵심인 UseCase 인터페이스를 통한 의존성 역전에 집중합니다. 완전한 아키텍처보다는 실습에 필요한 개념만 적용합니다.

3.3 UseCase 인터페이스 + 도메인 캡슐화 도입

UseCase 인터페이스가 왜 필요한지 이해했으니, 이제 실제 코드에 적용해보겠습니다. 패키지 구조부터 손보겠습니다. 챕터 2의 단순 레이어드 구조에서 `domain` 패키지를 안쪽으로 옮깁니다.

3.3.1 UseCase 인터페이스 정의

주문·상품·회원·배달 서비스의 각 기능을 인터페이스 형태로 UseCase로 정의합니다. 인터페이스 하나가 하나의 행위(Use Case)를 담습니다.

usecase/CreateOrderUseCase.java

```
public interface CreateOrderUseCase {
    OrderResponse createOrder(int userId, int productId, int quantity, Long price, String address);
}
```

3.3.2 엔티티의 비즈니스 로직 — DDD의 핵심

"주문이 취소 가능한가?" 같은 비즈니스 규칙은 서비스가 아닌 엔티티에 둡니다. 엔티티 메서드로 캡슐화하면 어디서 호출하던 동일한 규칙이 적용되고, 새 규칙이 들어와도 도메인 메서드만 손대면 됩니다.

domain/Order.java. validateCancelable 추가

```
public class Order {
    // 챗터 2 Order.java 참조 - 필드.create().complete() 동일, cancel()은 validateCancelable() 호출 추가

    // 비즈니스 규칙을 엔티티에 위임 (챗터 3에서 추가)
    public void validateCancelable() {
        if (this.status == OrderStatus.CANCELLED) {
            throw new Exception400("주문이 이미 취소되었습니다.");
        }
    }
}
```

3.3.3 OrderService - 인터페이스 구현

OrderService는 세 UseCase 인터페이스를 구현하고, 내부에서 도메인 객체의 비즈니스 메서드를 호출합니다. 이 구조 덕분에 책임이 분리됩니다. **서비스는 흐름 조율에만 집중하고, 실제 비즈니스 규칙은 도메인이 담당합니다.**

챗터 2와 비교해서 달라진 점은 다음과 같습니다.

1. **UseCase 인터페이스 구현.** 서비스가 직접 메서드를 노출하지 않고, `CreateOrderUseCase` 등 인터페이스를 구현합니다.
2. **비즈니스 규칙을 엔티티에 위임.** 챗터 2에서 서비스의 `if` 문으로 처리하던 검증을 도메인 객체의 메서드로 이동합니다.

usecase/OrderService.java. UseCase 인터페이스 구현

```
@RequiredArgsConstructor
@Service
@Transactional(readOnly = true) // 1. 클래스 레벨 읽기 전용 트랜잭션
public class OrderService implements CreateOrderUseCase, GetOrderUseCase, CancelOrderUseCase {
    // 2. UseCase 인터페이스를 구현

    @Override
    @Transactional // 쓰기 메서드만 오버라이드
    public OrderResponse cancelOrder(int orderId) {
        // ...
        findOrder.cancel(); // 3. cancel() 내부에서 검증 후 취소
        // ...
    }
}
```

3.3.4 OrderController 수정

`OrderService` 코드가 아닌 인터페이스를 주입받도록 컨트롤러를 수정합니다. 앞으로 `OrderService`를 다른 코드로 바꿔도 이 컨트롤러는 전혀 수정하지 않아도 됩니다.

web/OrderController.java. UseCase 인터페이스 주입

```
@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {
    private final CreateOrderUseCase createOrderUseCase; // 구현체가 아닌 인터페이스 주입
    private final GetOrderUseCase getOrderUseCase;
    private final CancelOrderUseCase cancelOrderUseCase;

    @PostMapping
    public ResponseEntity<?> createOrder(...) {
        return Resp.ok(createOrderUseCase.createOrder(...)); // 인터페이스 메서드 호출
    }

    // GET /{orderId} - 주문 조회
    // PUT /{orderId} - 주문 취소
}
```

order-service와 동일한 패턴으로 product-user-delivery 서비스도 UseCase 인터페이스를 도입하고, 검증 로직은 각 엔티티 메서드로 모읍니다.

코드 정리는 마쳤지만, 막상 운영에 올리려니 새 문제가 보입니다.

오픈이 : "이제 운영에 올려야겠죠? 그런데 네 서비스 포트가 흩어져 있어요. :8081 · :8082 · :8083 · :8084 . 사용자가 다 알아야 하나요?"

선배 : "내부를 수정했으니, 외부에서 들어오는 단일 진입점도 만들면 좋겠네요."

3.4 Docker - Gateway와 MySQL 인프라 이미지

3.4.1 Nginx - API Gateway 라우팅

서비스가 늘어날수록 클라이언트가 모든 포트를 알아야 하고, 서비스 주소가 바뀌면 클라이언트도 따라 바뀌어야 합니다. **API Gateway**를 앞에 두면 클라이언트는 하나의 진입점으로만 요청하고, 게이트웨이가 URL 경로를 보고 알맞은 서비스로 넘깁니다.

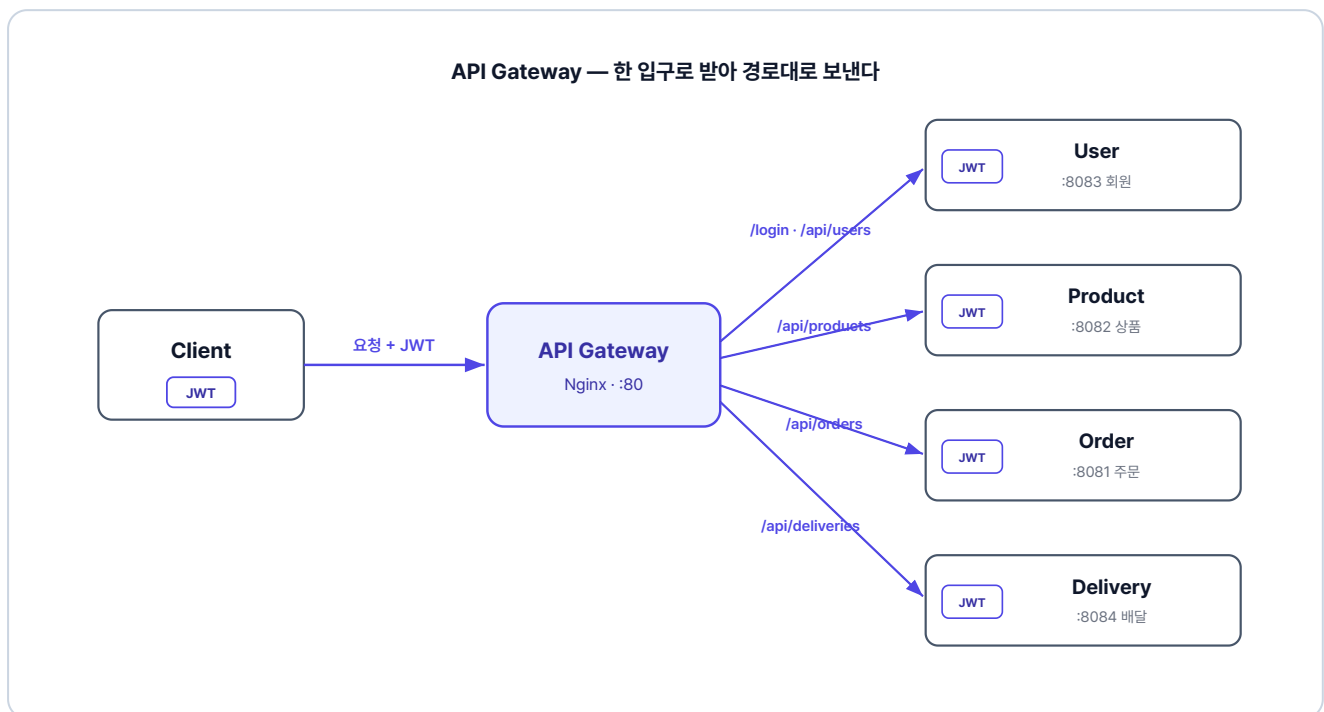


그림 3-5. 한 입구로 받아 경로대로 전달하고, 토큰 검증은 각 서비스가 합니다

`gateway/` 디렉토리에는 두 파일이 있습니다.

파일	역할
Dockerfile	Nginx 베이스 이미지에 설정 파일을 넣어 게이트웨이 컨테이너를 만듭니다.
nginx.conf	URL 경로별로 어느 서비스에 요청을 보낼지 정의합니다.

`nginx.conf`의 핵심은 네 서비스를 이름으로 등록하고, 들어온 URL을 해당 서비스로 보내는 것입니다. Kubernetes가 서비스 이름을 내부 DNS로 자동 해석하므로 IP 주소를 직접 지정하지 않아도 됩니다. 전체 설정 파일은 레포의 `gateway/nginx.conf`를 참고하세요.

게이트웨이에서 토큰을 검증하는 방식

지금 이 책에서는 게이트웨이(Nginx)가 요청을 경로대로 넘기지만 하고, JWT는 각 서비스가 직접 검증합니다. 다른 방법으로, 게이트웨이가 입구에서 토큰을 먼저 검증하고 통과한 요청만 서비스로 보내는 구조도 널리 쓰입니다. 이때 게이트웨이는 토큰에서 꺼낸 사용자 정보를 헤더에 실어 전달하고, 뒤 서비스는 게이트웨이를 지나온 요청을 이미 인증된 것으로 신뢰합니다.

이렇게 하면 인증 검사가 한 곳에 모여 각 서비스는 비즈니스 로직에만 집중하고, 인증 정책이 바뀌어도 게이트웨이만 고치면 됩니다. 대신 인증 책임이 게이트웨이에 몰리고, 게이트웨이를 우회한 내부 호출은 막지 못합니다. Nginx만으로는 토큰 검증이 안 되어 Spring Cloud Gateway 같은 도구를 쓰며, 이 책은 단순함을 위해 서비스별 검증을 씁니다.

3.4.2 MySQL - 데이터베이스 인프라

모든 서비스가 동일한 MySQL 인스턴스(`db-service:3306`)의 `metadb` 데이터베이스를 공유합니다. 서비스별로 테이블이 분리되어 있으나, 물리적으로는 단일 DB 인스턴스입니다.

실제 MSA에서는 서비스마다 독립된 DB를 둡니다. 이 책에서는 학습 편의를 위해 하나의 MySQL을 공유합니다. 학습 흐름을 익히는 데는 차이가 없으니 DB 구성보다 흐름에 집중해 주세요.

DB 컨테이너는 `db/` 디렉토리의 두 파일로 구성됩니다.

파일	역할
Dockerfile	MySQL 공식 이미지를 베이스로 초기화 SQL을 컨테이너에 넣어 띄웁니다.
init.sql	네 서비스에 필요한 테이블을 만들고 더미 데이터를 채웁니다.

3.5 Kubernetes - YAML로 선언하는 배포

3.5.1 매니페스트 구조 설계

Kubernetes는 YAML 파일로 원하는 상태를 선언합니다. "이 서비스는 이렇게 실행되어야 한다" 고 파일에 적어두면, K8s가 그 상태를 유지합니다.

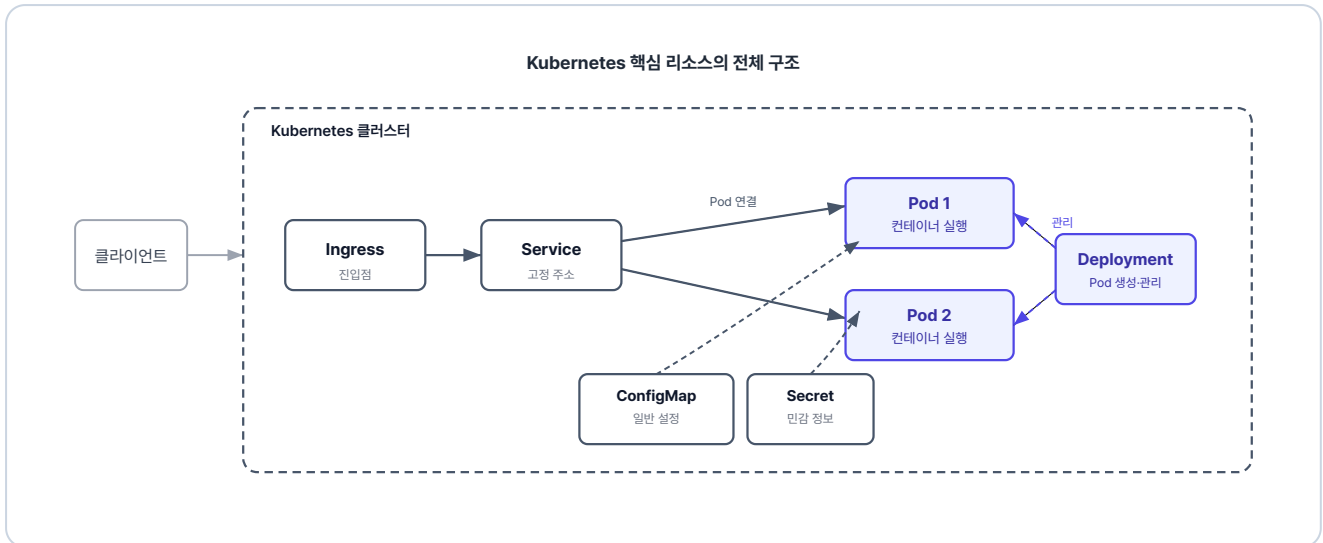


그림 3-6. Kubernetes 리소스 관계

각 K8s 리소스의 역할을 정리하면 다음과 같습니다.

리소스	역할
ConfigMap	일반 환경변수(DB 주소 등)를 외부에서 주입합니다.
Secret	민감 정보(DB 계정·비밀번호)를 분리해 관리합니다.
Secret (DB)	MySQL 컨테이너가 시작될 때 데이터베이스와 계정을 자동으로 만들 수 있도록 환경변수를 줍니다.
Deployment	Pod를 어떻게 실행할지 정의하고, ConfigMap과 Secret을 한꺼번에 주입합니다.
Service	Pod에 고정 주소를 부여해 클러스터 안에서 안정적으로 통신할 수 있게 합니다.
Ingress	클러스터 외부 요청을 클러스터 안으로 들여보냅니다.

나머지 서비스(product, user, delivery)도 동일한 패턴입니다. 전체 매니페스트는 레포의 [k8s/](#) 디렉토리를 참고하세요.

Gateway API로 두면 라우팅 규칙을 재사용하기 쉽다

이 책은 minikube에서 Ingress로 외부 진입을 구성하지만, Ingress의 후속 표준인 Gateway API를 쓰는 선택지도 있습니다. 쿠버네티스에 기본 내장된 Ingress와 달리 이 표준은 별도로 설치해 쓰며, 진입점을 정의하는 Gateway와 경로 규칙을 정의하는 HTTPRoute로 역할이 나뉩니다. 경로 규칙은 환경이 바뀌어도 그대로 두고, 진입점 정의만 환경에 맞춰 바꿉니다.

AWS EKS 같은 환경에서는 컨트롤러가 이 선언을 읽어 관리형 로드 밸런서를 붙여 줍니다. 로컬에서는 진입점을 직접 띄워야 하지만, 클라우드에서는 이 과정이 자동으로 처리됩니다. 라우팅 규칙을 다시 짤 필요가 없어, 로컬에서 쓰는 경로 설정을 운영 환경에서도 그대로 씁니다.

3.6 Minikube - 실행 및 결과 확인

3.6.1 Minikube 시작

Minikube는 로컬 PC에 가벼운 Kubernetes 클러스터를 만들어주는 도구입니다. 설치되어 있지 않다면 OS에 맞게 먼저 설치합니다.

터미널 Minikube 설치 - macOS

```
brew install minikube
```

터미널 Minikube 설치 - Windows

```
winget install Kubernetes.minikube
```

설치한 뒤 새 터미널을 열고, Docker Desktop이 실행 중인 상태에서 아래 명령을 입력하면 클러스터가 생성됩니다.

터미널 Minikube 시작

```
minikube start
```

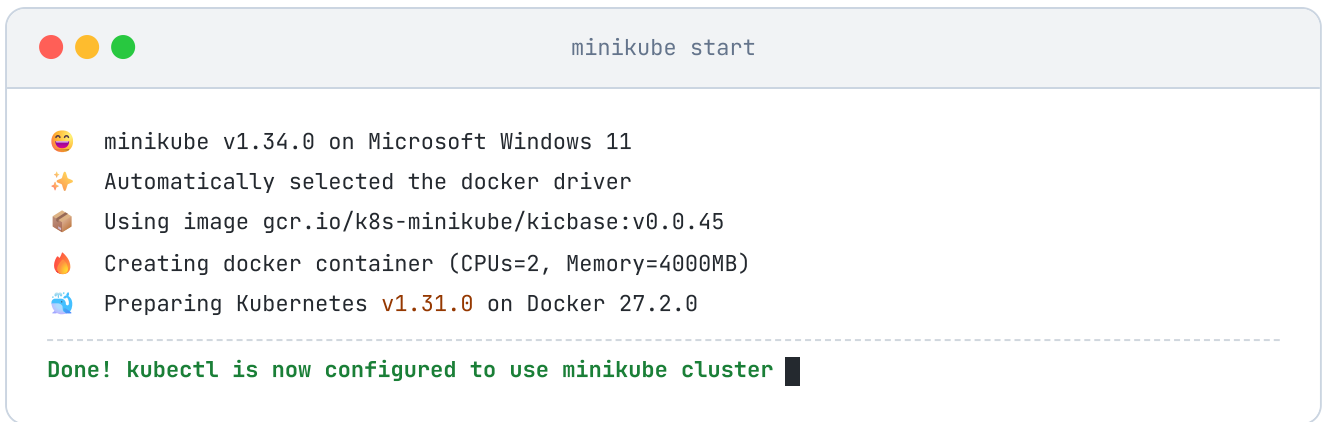


그림 3-7. Minikube 시작

처음 실행하면 필요한 이미지를 다운로드하므로 몇 분 정도 걸릴 수 있습니다.

3.6.2 이미지 빌드

`minikube image build` 는 Minikube 내부에 직접 이미지를 빌드합니다.

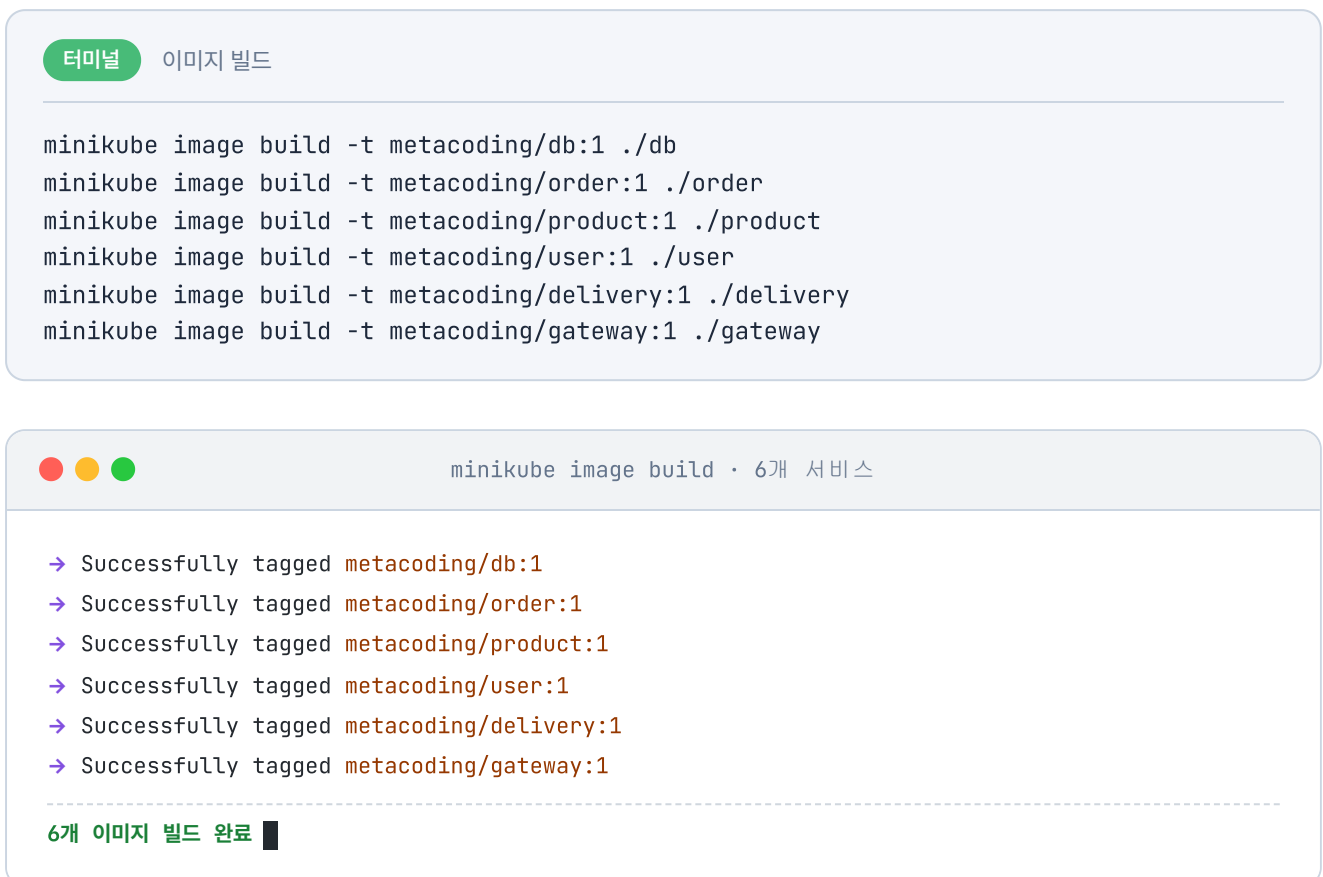


그림 3-8. 이미지 빌드 결과

3.6.3 배포 순서

네임스페이스를 먼저 생성하고, DB가 준비된 뒤에 나머지 서비스를 배포합니다.

터미널 배포 순서

```
# 1. 네임스페이스 생성
kubectl create namespace metacoding

# 2. DB 관련 리소스 먼저 배포
kubectl apply -f k8s/db

# 3. 각 서비스 배포
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
kubectl apply -f k8s/delivery
kubectl apply -f k8s/gateway

# 4. Ingress 활성화 (Minikube에서는 애드온 활성화 필요)
minikube addons enable ingress
```

kubectl apply · 네임스페이스 + 6개 서비스

```
namespace/metacoding created
secret/db-secret created
deployment.apps/db-deploy created
service/db-service created
configmap/order-configmap created
secret/order-secret created
deployment.apps/order-deploy created
service/order-service created
... product · user · delivery · gateway 동일 패턴 ...
ingress.networking.k8s.io/gateway-ingress created

-----
전체 리소스 배포 완료 ■
```

그림 3-9. 네임스페이스 생성 및 배포

3.6.4 배포 상태 확인

배포가 끝나면 모든 Pod가 제대로 실행되고 있는지 확인합니다.

터미널 Pod 상태 확인

```
kubectl get pods -n metacoding
```



kubectl get pods -n metacoding

NAME	READY	STATUS	AGE
db-deploy-6f9b7c4d8-m4t2q	1/1	Running	42s
gateway-deploy-5c8d6f7b9-h7w3r	1/1	Running	38s
order-deploy-8b7f6c9d4-q2k8m	1/1	Running	36s
product-deploy-7c9d8b6f5-x4r2t	1/1	Running	35s
user-deploy-6d8c7b9f4-p3m9k	1/1	Running	33s
delivery-deploy-9f7c8b6d5-t6w2x	1/1	Running	30s

모든 Pod Running ■

그림 3-10. Pod 상태 확인

모든 Pod가 **Running** 상태가 되면 배포 완료입니다.

3.6.5 서비스 접근

Ingress를 통해 외부에서 접속하려면 **minikube tunnel** 을 실행합니다.

터미널 외부 접근 터널

```
minikube tunnel
```

minikube tunnel 은 터미널을 점유합니다.

터널이 실행되면 **http://127.0.0.1:80** 로 gateway-service에 접속할 수 있습니다. 회원 서비스가 발급한 토큰은 하루 동안 유효하므로, 챕터 2에서 받은 토큰을 그대로 써서 아래 주문을 생성합니다. 만료됐다면 같은 방법으로 다시 발급받습니다.

POST <http://127.0.0.1:80/api/orders>

```
{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": "Addr 4"
}
```

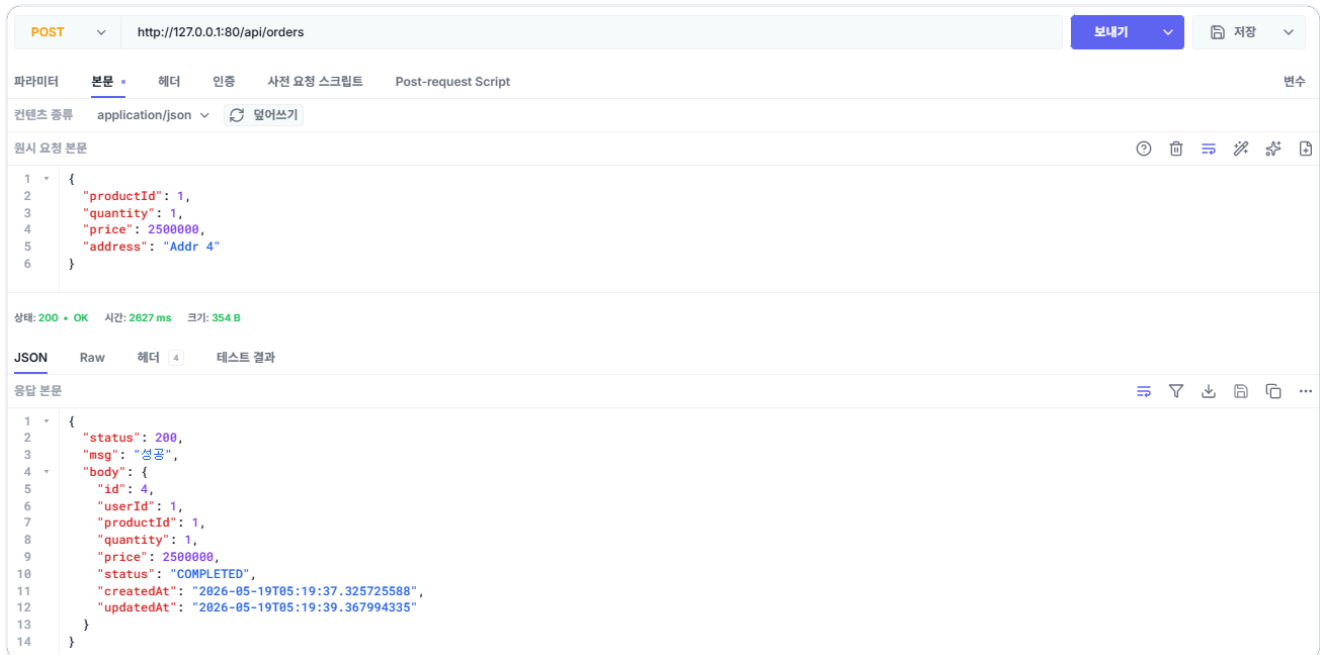


그림 3-11. 주문 결과 확인

테스트가 끝났으면 이번 챕터에서 실행한 리소스를 정리합니다.

터미널 리소스 정리

```
kubectl delete namespace metacoding
```

오픈이 : "Kubernetes에서도 주문이 잘 되네요! 안은 도메인이 자기 일을 알고, 밖은 한 입구, 위에서는 K8s가 살려주잖아요."

선배 : "K8s가 Pod 죽으면 살려주죠. 그런데 어떤 Pod이 죽었다가 다시 살아나는 그 사이에는 어떻게 될 것 같아요?"

살아나는 동안에는...?

오픈이가 답을 못 했습니다. K8s가 죽은 Pod을 알아서 다시 띄워준다니 괜찮을 것 같기도 하고, 그런데 다시 뜨는 그 사이에 들어온 요청이 어떻게 될지는 아직 떠올려 본 적이 없었습니다.

코드 구조도 좋아졌고, 운영 배포도 됐고, `Order`가 자기 일을 답하게 됐습니다. 다만 선배의 질문이 머릿속에 남았습니다.

이것만은 기억하자

→ **DDD**로 비즈니스 규칙을 도메인 객체에 캡슐화했습니다.

`Order.validateCancelable()` · `Order.cancel()` 로 도메인이 자기 일을 압니다.

→ **클린 아키텍처**(UseCase 인터페이스)로 컨트롤러가 `OrderService` 코드가 아닌 약속에 의존하게 했습니다. 환경별·테스트별로 `OrderService` 코드를 갈아끼울 수 있습니다.

→ **Nginx API Gateway**로 네 서비스 진입점을 하나로 모았습니다. URL 경로에 따라 적절한 서비스로 라우팅됩니다.

→ **Kubernetes 배포**에서 ConfigMap·Secret으로 환경변수를 주입하고, Deployment가 Pod를 자동으로 복구합니다.

다음 챕터에서는 동기 호출의 한계를 정면으로 마주칩니다. Kafka로 서비스 간 전화를 끊고, 메시지를 사이에 두며, 중앙에 지휘자(orchestrator)를 둡니다. 한 서비스가 잠시 멈춰도 전체 시스템은 계속 동작합니다.